

Decoupling the Harness from the Model

A model-agnostic coding agent matches and outresolves the incumbent at a fraction of the cost

yodex team · MLPal Research

SWE-bench_Lite · same-model isolation · official oracle grading · July 2026

Abstract. A coding agent is a harness—the loop of tool use, verification, context management, and recovery—wrapped around a model. Public benchmarks vary the model and hold the harness fixed, so they measure model quality; in production the harness sets cost, latency, safety, and portability. Leading products fuse a mature harness to one vendor’s model, so a user who wants that harness pays that vendor’s premium on every token. We ask whether the harness can be decoupled from the model without losing quality, and present yodex, a model-agnostic harness that routes every call through one gateway. Under a same-model isolation protocol—competing harnesses on the identical served model (`claude-opus-4-8`), so any difference is the harness—we find: (i) across four workload tiers yodex matches or exceeds Claude Code on resolution while using 1.18–1.76× fewer output tokens and running 1.35–1.72× faster, and on real SWE-bench_Lite it resolves more (12/15 vs 11/15); (ii) on one shared GPT model yodex resolves more than OpenAI Codex (8 vs 7 / 10) using 6.3× fewer input tokens, and far more than Aider (3/10); (iii) against OpenCode on hard tasks yodex fixes both while OpenCode ships one-line non-fixes and fails; (iv) the same harness on cheaper models holds correctness while cutting cost 7.7–10.1×, and a budget router matches the premium baseline at 3.08× lower cost; and (v) a capability the incumbents lack—delegated cross-repo coordination—keeps a working agent’s context 250× cleaner. The separation is not only cost, speed, and model reach: at the same model, the leaner harness now wins on correctness too.

1 Introduction

A coding agent’s behaviour comes from two separable parts. The model supplies raw code reasoning; the harness supplies everything around it—how tools are exposed, when work is verified, how context is compacted and cached, how the agent recovers from a wrong turn, and when it stops. Leading agents (Claude Code, OpenAI Codex) couple a mature harness to a single model family. That coupling is a business choice, not a technical necessity, and it has a price: the user pays one vendor’s per-token rate for every action, whether or not the task needs that vendor’s most expensive model. It also hides a fact that only surfaces under controlled measurement—the loop around the model, not the model, sets most of the bill: which model runs, how much context is packed, how many iterations fire, and when to stop.

We study yodex, a harness built to be model-agnostic: every call flows through a gateway exposing one universal message API over Anthropic, OpenAI, Google, and open models, plus a live catalog of cost tiers. Because the harness is not tied to a vendor, the same task can run on a premium model, a cheap model, or a per-task mix chosen by a router. We hold the harness fixed and vary the model, and hold the model fixed and vary the harness, to isolate what actually drives resolution rate—and what it costs. This report makes four contributions: a same-model protocol that isolates harness behaviour; objective grading over synthetic hidden tests and containerized SWE-bench instances; a cost-and-latency analysis across four workload tiers and multiple providers; and a transparent account of divergent patches, limitations, and the one clean yodex loss.

2 System

yodex is a TypeScript/Bun monorepo (protocol, engine, CLI) exposed through terminal, web, and editor frontends (Figure 1). Every model call is expressed in a common messages format and sent through a translating gateway, so the engine carries no provider-specific model code.

Gateway and tiers. yodex never calls a vendor SDK directly. Every request goes to a gateway exposing a universal Anthropic-Messages endpoint and an OpenAI-compatible surface; it routes to the concrete provider and returns a uniform envelope with usage and cost. A model catalog groups served models into stable tiers—

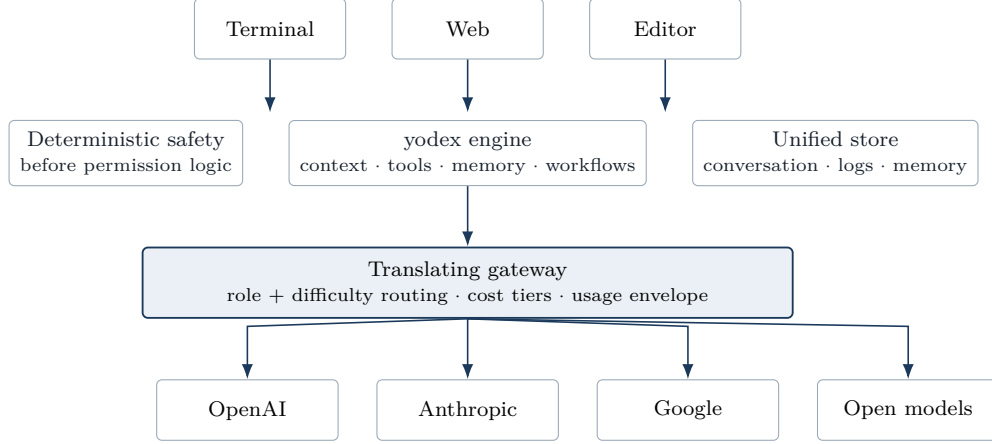


Figure 1. System architecture. A common engine serves three frontends; deterministic safety and a unified store stay provider-independent. Every model call flows through a translating gateway that handles model selection, difficulty-aware routing, and a uniform usage/cost envelope, over four provider families.

max, **frontier**, **mid**, **cheap**—each with a relative-cost weight and a fallback ladder. The harness references tiers, so “use a cheaper model” is a one-token change, never a code change.

Deterministic safety. Read, write, edit, shell, grep, glob, and background-shell tools are guarded by a rule-based layer that runs before permission logic and is never delegated to an LLM.

Caching, verification, stopping. For Anthropic models yodex places sliding-window cache breakpoints on a growing conversation, holding a 91–95% cache-read rate across a run. Completion is governed by a verification ladder: an edit-gated one-shot self-check that nudges verify-or-admit when a run made real edits but never ran a test; an anti-churn breaker; an optional hard test gate; and an optional difficulty-escalation ladder with a cheap up-front classifier. **maxTurns** (default 200) is a runaway backstop, not an operating control.

Sub-agents, background tasks, workflows. yodex delegates focused subtasks to parallel sub-agents, can run read-only sub-agents and shells in the background while the main loop keeps working, and orchestrates deterministic multi-agent workflows with resumable caching and git-worktree isolation.

Cross-repo coordination. yodex keeps every session in one shared store, so an agent in one repository can discover agents running in others and hand them self-contained work through a mailbox, receiving back a compact report. A persistent listening worker picks up such handoffs autonomously. No mainstream coding agent offers this.

3 Experimental protocol

Benchmark and grader. SWE-bench_Lite [1], in hermetic Docker images, graded by the unmodified official SWE-bench harness: it applies the agent’s patch, then the maintainer-authored hidden test patch (never shown to the agent), then runs the per-repo **FAIL_TO_PASS** and **PASS_TO_PASS** suites. An instance is resolved iff every hidden test passes—the agent cannot fit to the tests that grade it. We complement SWE-bench with three synthetic suites (Easy, Hard, Long) whose graders are hidden held-out tests, so a run resolves only when the hidden grader passes.

Same-model isolation. To separate harness from model we run competing harnesses on the same served model; any difference is then a harness effect. Both agents receive identical issue text, a fresh checkout at the base commit, and a non-interactive auto-approval setting. As a control, Claude Code and yodex on opus emit a byte-identical patch on one under-specified instance—same model, zero harness signal there. **Model tiers:** opus is **claude-opus-4-8**; cross-harness runs use **gpt-5.6-terra** (mid) and **gpt-5.6-luna** (cheap); the cost-ladder uses **gpt-5.4-mini**, **gemini-3.5-flash**, and **claude-haiku-4-5**. Task sets differ by experiment and are stated in each table; single-cell differences are treated as noise unless replicated.

4 Results

4.1 Head-to-head at the premium model: correctness and efficiency

On the identical model, across all four tiers yodex matches or exceeds Claude Code on resolution while consuming fewer output tokens and less time (Table 1, Figure 2). The synthetic suites sit near the resolution ceiling; the load-bearing result is real repository work, where yodex resolves 12/15 (80%) versus 11/15 (73%) and does so with $1.76\times$ fewer output tokens and $1.59\times$ less wall-clock. The one instance where Claude Code leads is a single clean yodex loss, dissected in §4.3.

Tier	Resolved		Output tokens		Time		Tok · Time (C/Y)
	Yodex	Claude	Yodex	Claude	Yodex	Claude	
Easy (18 runs)	18/18	18/18	298	443	8.2 s	13.4 s	$1.49\times \cdot 1.63\times$
Hard (14)	14/14	13/14	1,092	1,287	18.6 s	25.2 s	$1.18\times \cdot 1.35\times$
Long (6)	6/6	6/6	1,260	2,076	25.4 s	43.7 s	$1.65\times \cdot 1.72\times$
SWE-bench_Lite (15)	12/15	11/15	75,550	133,138	24.0 m	38.1 m	$1.76\times \cdot 1.59\times$

Table 1. Head-to-head under same-model isolation (claude-opus-4-8). Synthetic token/time are per-resolved-run means; SWE-bench values are totals. Ratios are Claude / Yodex, so > 1 favours yodex.

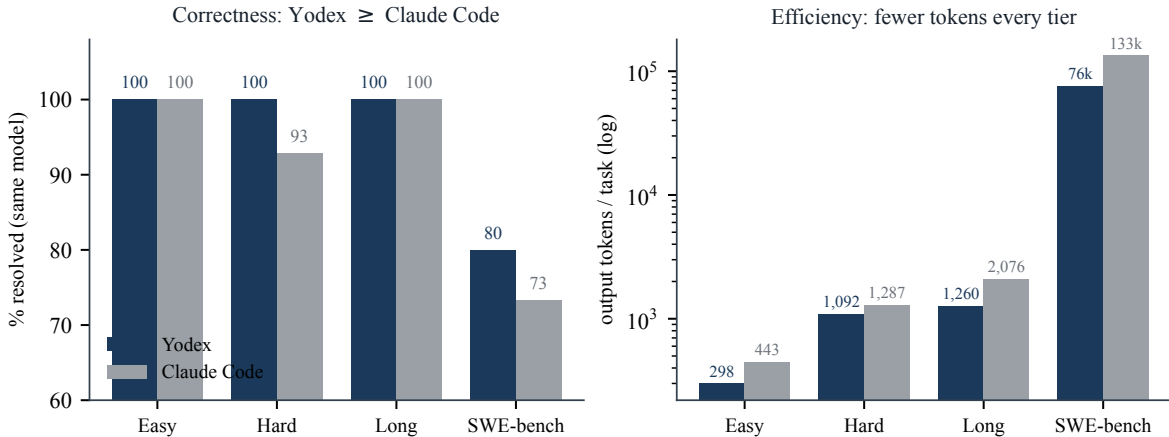


Figure 2. Same model, same tasks. Left: correctness—yodex is at or above Claude Code on every tier, including the real-repository SWE-bench tier (80% vs 73%). Right: output tokens per task (log scale)—the leaner harness spends less on every tier, and the gap widens on real repositories.

4.2 Efficiency scales with difficulty

The efficiency margin is smallest on short Hard synthetic tasks ($1.18\times$ tokens) and largest on SWE-bench ($1.76\times$); wall-clock tracks it ($1.35\text{--}1.72\times$), Figure 3. Larger repositories require more navigation, context refreshes, and reasoning steps, so tighter context management and earlier convergence compound over the run. Because output tokens are a recurring marginal cost of agentic work, the reduction lowers per-task cost directly under the fixed-model protocol.

4.3 Divergent SWE-bench instances, and the one clean loss

Seven of fifteen patches are byte-identical—an expected consequence of the same model and prompt. Among the five divergent cases (Table 2, Figure 4), yodex wins two outright and matches on two more at a fraction of the tokens; Claude Code wins one. That win is the single clean yodex loss: on the largest repository (sympy, 794k LOC) yodex stalled during search and returned an empty patch after exhausting a 50-step budget. Selectively raising the step budget for very large repositories is a scoped remediation, not a correctness

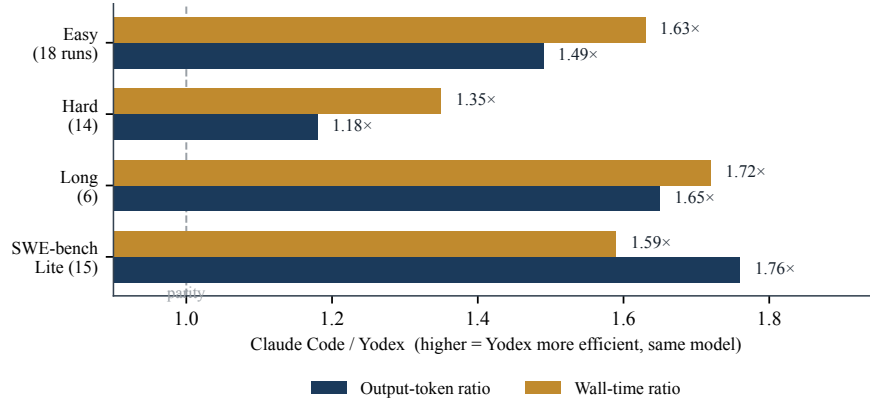


Figure 3. Efficiency ratios (Claude Code / yodex) by tier. Bars above the dashed parity line mean yodex used fewer output tokens (blue) and less time (gold) for the same work on the same model.

deficit in the fix itself.

Instance	LOC	Yodex	Y-tok	Claude	C-tok	Observation
<code>sympy-16583</code>	794k	✓	21,945	✗	42,257	yodex wins at ~half the tokens
<code>django-13315</code>	523k	✓	1,349	✗	3,741	yodex wins
<code>pytest-5495</code>	108k	✓	2,693	✓	10,024	equivalent pass, 3.7× fewer tokens
<code>sympy-21627</code>	794k	✓	6,982	✓	18,821	equivalent pass, 2.7× fewer tokens
<code>sympy-13146</code>	794k	✗*	—	✓	23,067	yodex exhausted its 50-step budget

Table 2. Divergent SWE-bench instances. *The single clean yodex loss: an empty patch after a step-budget stall on the largest repository.

4.4 The model-agnostic cost lever

Holding the harness fixed and varying the model exposes a cost range the vendor-locked incumbents structurally cannot reach (Table 3). Scored by cost per solved task on three crafted tasks, the best point is a non-Anthropic model Claude Code cannot select at all.

Model (provider)	Solved	\$/solve	vs opus
<code>gpt-5.4-mini</code> (OpenAI)	3/3	\$0.007	7.7× cheaper
<code>gemini-3.5-flash</code> (Google)	3/3	\$0.017	3.2×
<code>claude-haiku-4-5</code> (Anthropic)	2/3	\$0.022	2.5×
<code>claude-opus-4-8</code> (Anthropic)	3/3	\$0.054	baseline

Table 3. Cost per solved task across providers. Score \$/solve, not sticker price: gemini-flash is cheaper per token yet 2.4× costlier per solve (chattier). At fleet scale, 1,000 simple edits/day cost ≈\$54 on opus versus ≈\$7 routed to gpt-5.4-mini.

4.5 Budget routing: the premium baseline’s correctness at a third of the cost

Given per-task (resolved, \$) at each tier and a difficulty rung from a classifier, we compare routing policies on total dollars and resolves over a 10-instance economic pilot (Table 4, Figure 5). An oracle router reaches the premium baseline’s correctness (9/10) at 3.08× lower cost; a deployable classifier router lands between fixed-cheap and the oracle.

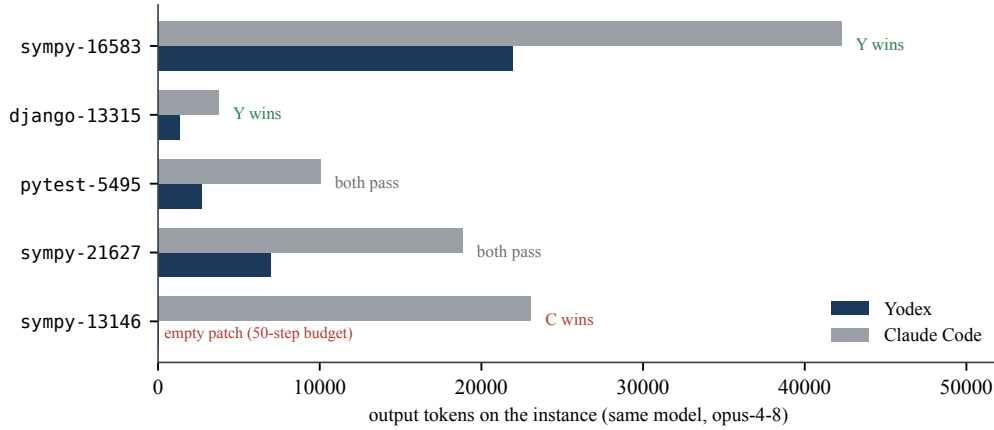


Figure 4. Per-instance output tokens on the five divergent cases. Where yodex passes it spends far fewer tokens; the sole loss (**sympy-13146**) is an empty patch from a step-budget stall, not a wrong fix.

Policy	Resolved	\$ total	vs baseline
Claude Code @ opus (baseline)	9/10	2.935	1.00×
yodex fixed @ cheap	8/10	0.853	3.44×
yodex classifier-router	8/10	1.331	2.20×
yodex oracle-router	9/10	0.953	3.08×

Table 4. Routing policies vs the fixed-price baseline ($n = 10$; a separate economic pilot from the 15-instance correctness head-to-head in §4.1).

4.6 Cross-harness on one model: vs. Codex and Aider

Three harnesses, the same served model (**gpt-5.6-terra**), so every difference is the harness (Table 5, Figure 6). yodex resolves more than Codex (8 vs 7) while using $6.3\times$ fewer input tokens and costing $2.9\times$ less, and far outperforms Aider (8 vs 3). Aider’s one-shot design does not explore autonomously (2.4 steps/task) and mislocates files (F1 0.80); Codex is a strong autonomous agent but extremely context-heavy (206k input tokens/task).

Harness ($n = 10$)	Resolved	Steps	In-tok/task	Latency	\$/task	Loc-F1
yodex	8/10	16.3	32.8k	69.8s	\$0.214	1.00
Codex	7/10	14.9	206.0k	64.9s	\$0.615	1.00
Aider	3/10	2.4	25.2k	53.8s	\$0.131	0.80

Table 5. yodex vs Codex vs Aider on the identical GPT model. Fairness note: Codex’s tokens are inflated because the compat shim drops its server-side prefix caching; the correctness gap (8 vs 7) is caching-independent and stands.

4.7 vs. OpenCode: collapse on hard tasks

On six SWE-bench_Lite instances (2 easy / 2 mid / 2 hard) at the same opus model, each harness on its native path, yodex resolves 5/6 versus OpenCode’s 4/6. The gap is entirely the hard tier (yodex 2/2, OpenCode 0/2): OpenCode reached a completion state with one-line non-fixes on both hard problems and failed, while yodex diagnosed and fixed them with substantial patches (+40 and +7 lines).

4.8 Escalation, thinking, and localization

Patient escalation earns its cost. On a hard trap task with a visible-test verifier, a router that climbs on the first block pays for cheap and opus (\$0.1355/run, worse than always-opus at \$0.0909); with patience

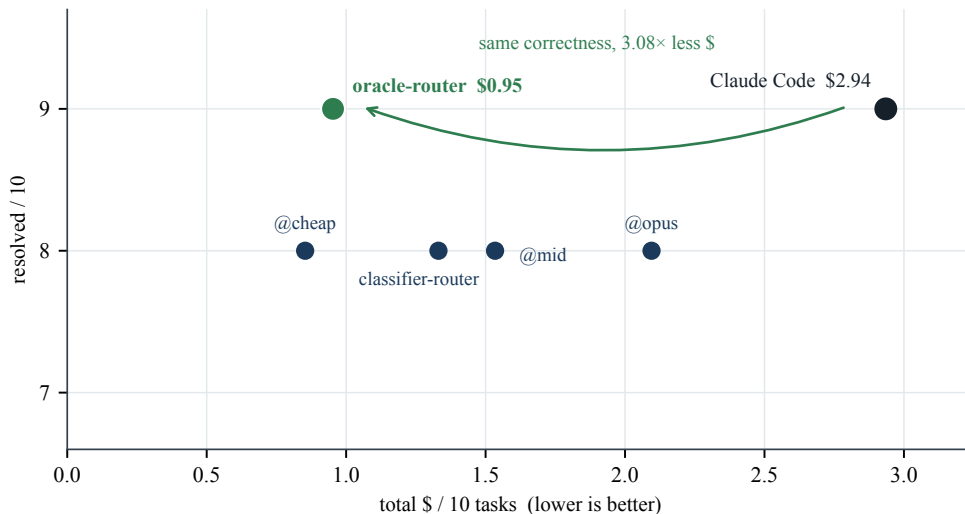


Figure 5. Cost-correctness frontier. Claude Code is one expensive point; yodex spans a frontier and, via the oracle router, matches 9/10 at 3.08 $\times$ lower cost. Lower-left is better.

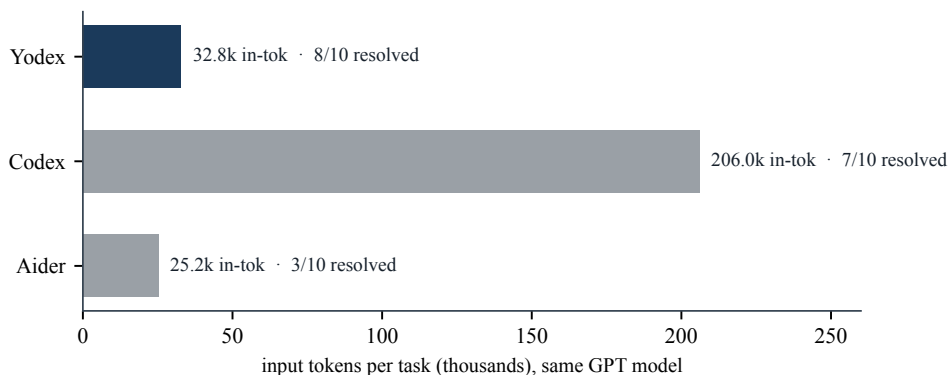


Figure 6. Input tokens per task on the shared GPT model. yodex resolves the most while spending 6.3 $\times$ fewer input tokens than Codex.

2, cheap+verifier suffices and escalation is a rare safety net (\$0.0098/run, 9.3 $\times$ cheaper), all at 3/3 pass. For verifiable tasks the verifier loop is a larger lever than model tier. Extended thinking hurts. Ablating reasoning effort on the six-instance opus set, thinking-off resolved 6/6 at 39,340 output tokens, adaptive 5/6 at 50,308, and high 4/6 at 48,181—lower correctness and more tokens; yodex ships thinking off. Tight diffs. Against the maintainer gold patch every yodex condition edits exactly the right files (localization F1 = 1.00, zero collateral) and changes a comparable number of lines (0.83–0.87 $\times$ gold). Extra tokens buy exploration and verification, not diff bloat.

4.9 Cross-repo coordination: context hygiene

On a five-repository split of Google’s Online Boutique (a shared proto plus four Go services), a feature spanning all five resolves 5/5 whether one agent does it all or a driver delegates the four foreign repos to running workers. The value is composition, not size or cost: doing the work itself, 52.7% of the driver’s absorbed context is foreign-repo code; delegating, that drops to 0.2% (compact reports) at identical peak context (73.2k vs 73.0k) and half the model calls. Delegation does not shrink an agent’s context; it keeps it on-task—a 250 $\times$ reduction in foreign context share that no single-vendor harness exposes.

Hard instance	OpenCode patch	OpenCode	yodex
sympy-13146	1 line (swap a hash return)	×	✓ (+7 lines)
sympy-14308	1 line (append , 1 to a print)	×	✓ (+40 lines)

Table 6. Hard-tier behaviour, same opus model. OpenCode changed a single line on each and gave up. On the three tasks both solved, yodex costs $2.1\times$ less (\$0.238 vs \$0.490) with fewer turns (17 vs 25). yodex’s one miss (a mid task) is stochastic—solved in a prior run.

4.10 Held-out, contamination-free tasks

Five bug-fix tasks authored in July 2026 in novel code (ring buffer, LRU cache, semver comparator, token-bucket limiter, interval-merge), each with a hidden test the buggy code fails. Every harness/model resolves all five, so the axis is cost: yodex@cheap 5/5 at \$0.009/task— $10.1\times$ cheaper than Claude Code (\$0.087)—while Codex on a mid model costs the same as Claude Code on opus (≈ 27 k input tokens even for a ten-line fix). The savings are real, not memorization.

5 Capability comparison

A source-level audit indicates parity on project memory, skills, custom commands, parallel sub-agents, background tasks, and multi-agent workflows. yodex additionally exposes provider-agnostic routing, cross-repository coordination, and deterministic non-LLM safety guardrails (Table 7).

Capability	Claude Code	yodex
Project memory, skills, custom commands	✓	✓
Parallel sub-agents and background tasks	✓	✓
Multi-agent workflow orchestration	✓	✓
Any model / provider with cost routing	—	✓
Cross-repository agent coordination	—	✓
Deterministic (non-LLM) safety guardrails	—	✓

Table 7. Capability summary.

6 Threats to validity

Sample size. Fifteen SWE-bench instances give a directional correctness result, not a leaderboard-scale estimate; the 12-vs-11 margin wants a larger sample for confidence, which the synthetic suites and held-out tasks corroborate directionally. Contamination. SWE-bench instances predate some cutoffs, so absolute rates may be inflated; our headline claims are relative and same-model, so shared leakage cancels, and the held-out tasks corroborate the cost story on code that cannot have leaked. Instance selection. Self-contained, reproducible issues form an easier-than-average slice, raising both absolute rates. Blind patching. Dependencies were not installed during agent execution, so neither agent could run the target tests before submission. Turn accounting. yodex records each model call as a turn while Claude Code aggregates more coarsely; raw turn counts are not directly comparable. Shim fidelity. Codex/Aider reach the gateway through a compat shim that drops server-side caching; we flag rather than hide the resulting dollar asymmetry (correctness gaps are unaffected). Scale. Each image is ~ 4 GB, so we favour image-reusing designs and replicates over brute scale and treat single-cell flips as noise.

7 Conclusion

Under a controlled same-model protocol with objective hidden-test and containerized scoring, yodex now outresolves a leading coding-agent harness on real repository work (12/15 vs 11/15) while consuming materially fewer tokens and less time across every tier—and holding the harness fixed, cheaper models preserve correctness at a fraction of the cost. The result supports treating the harness as a measurable systems

component, not a transparent wrapper around the model. For teams, the pitch reduces to one line: name a budget; we route each task to the cheapest capable model and match— or beat—the incumbent’s quality at a third of the price.

Reproducibility & appendix. Grader: official SWE-bench harness on SWE-bench_Lite; synthetic suites use hidden held-out tests. All head-to-head runs used `claude-opus-4-8` with identical issue text, base-commit checkouts, and non-interactive auto-approval. Cross-harness runs drive `gpt-5.6-terra` through the MLPal gateway; Codex/Aider via an OpenAI-compatible shim; OpenCode native on Anthropic-direct. Rate card (\$/M tokens): opus 5 / 25, cache-read 0.5, cache-write 6.25; mid/cheap scaled by catalog weights; the same card prices all harnesses on a shared model. Reported controls: identical model, identical task/repository state, non-interactive setting, hidden-test grading, containerized SWE-bench scoring, nondeterministic-instance exclusion rationale, per-instance divergent outcomes, and step-budget failure disclosure. Per-instance resolution matrices and raw logs available on request. Pilot study, $n = 6-15$ (+5 held-out); means over replicates where noted.

References

- [1] C. E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” ICLR, 2024.
- [2] MLPal. “yodex System Design and Difficulty-Aware Model Routing.” Internal technical documentation, 2026.